

algoritmos

Leslie Garcia

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos en R

El lenguaje R es un entorno de programación ampliamente utilizado en el campo de la estadística, el análisis de datos y la ciencia de datos. Fue desarrollado con el objetivo de facilitar el procesamiento, la manipulación y la visualización de grandes volúmenes de información de manera eficiente. Además, R es un lenguaje muy parecido a MathLab y trabaja con variables que en general suelen ser matrices.

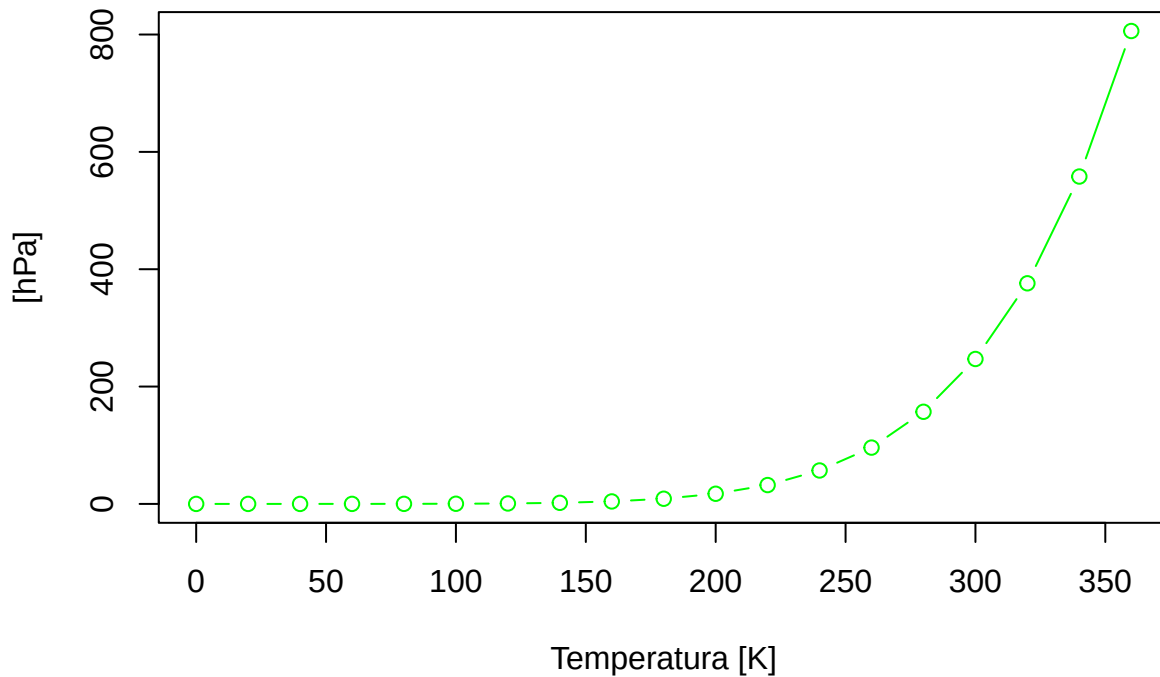
Una de las principales características de R es su capacidad para realizar análisis estadísticos avanzados, incluyendo modelos de regresión, pruebas de hipótesis y técnicas de aprendizaje automático. Además, cuenta con una gran variedad de paquetes y librerías que amplían sus funcionalidades, permitiendo abordar problemas complejos en distintas áreas como la economía, la ingeniería, la biología y las ciencias sociales.

R también se destaca por su potente sistema de visualización, que permite generar gráficos de alta calidad para representar datos de forma clara e intuitiva. Gracias a su carácter de software libre y su activa comunidad de usuarios, se ha convertido en una herramienta fundamental para investigadores, estudiantes y profesionales.

En la actualidad, el lenguaje R es una de las herramientas más importantes para el análisis de datos, siendo ampliamente utilizado en entornos académicos y profesionales.

```
plot(pressure,main="Presión del gas ideal",ylab="Presión  
[hPa]",xlab="Temperatura [K]",type="b", col="green")
```

Presión del gas ideal



```
A<-38
A<-40
B<-60
C<-B-A
C
```

```
## [1] 20
```

```
y=40
alpha<-30
```

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   : 2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

Uso de Data Set internas de R

Conclusión

El uso de R en **Posit Cloud** simplifica enormemente el acceso al análisis de datos, ya que elimina la necesidad de instalación y configuración local. Esto lo convierte en una herramienta ideal para estudiantes y trabajos colaborativos, permitiendo comenzar a programar de forma inmediata desde cualquier dispositivo.

Sin embargo, esta comodidad viene acompañada de ciertas limitaciones, como la dependencia de una conexión a internet y restricciones en los recursos computacionales. Por lo tanto, **Posit Cloud es excelente para aprendizaje, prototipado y proyectos medianos**, mientras que para tareas más pesadas o profesionales

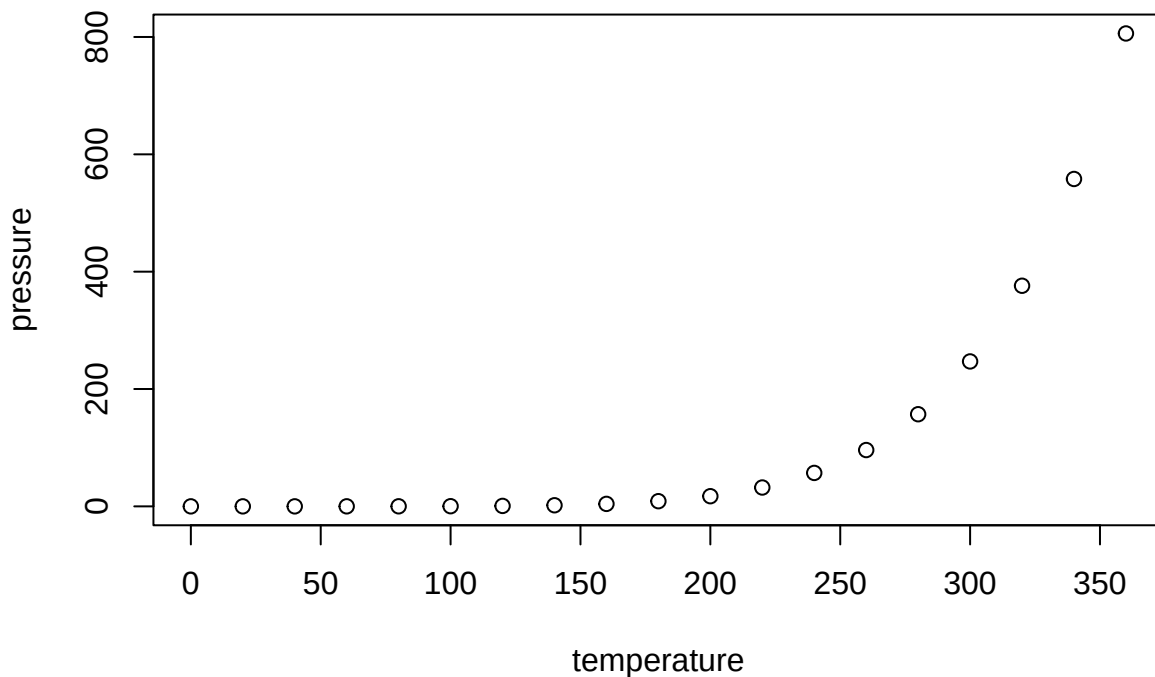
puede ser conveniente trabajar en un entorno local.

Tabla comparativa: Posit Cloud vs R local

Característica	Posit Cloud	R local
Instalación	No requiere	Requiere instalación
Acceso	Desde cualquier navegador	Solo en el dispositivo
Configuración inicial	Muy simple	Puede ser compleja
Uso de recursos	Limitado (según plan)	Depende del hardware propio
Conexión a internet	Obligatoria	No necesaria
Velocidad	Depende de internet	Generalmente más rápida
Trabajo colaborativo	Muy fácil (compartir proyectos)	Más difícil
Personalización	Limitada	Alta
Uso educativo	Excelente	Bueno
Proyectos grandes	Limitado	Ideal

Including Plots

You can also embed plots, for example:



Data Set en R

¿Qué es un dataset? Es un conjunto de datos organizado (tabla con filas y columnas) que se usa para análisis estadístico.

Tipos de datasets en R:

Internos → vienen incluidos en R o paquetes Ej: `iris`, `mtcars` **Externos** → archivos que importás Ej: CSV, Excel, TXT

Datasets internos

- Ya están disponibles sin descargar

- Se usan para aprender y practicar

Comandos básicos:

```
data() # lista todos los datasets
data(iris) # carga uno
```

Datasets externos

- Se cargan desde archivos **Ejemplos:**

```
read.csv("archivo.csv")
read.table("archivo.txt")
```

```
head(datos) # primeras filas
summary(datos) # resumen estadístico
str(datos) # estructura
```

Exploración de datos

Idea clave Los datasets son la base del análisis en R:

sin datos, no hay análisis. Permiten aplicar estadísticas, visualizar información y tomar decisiones.

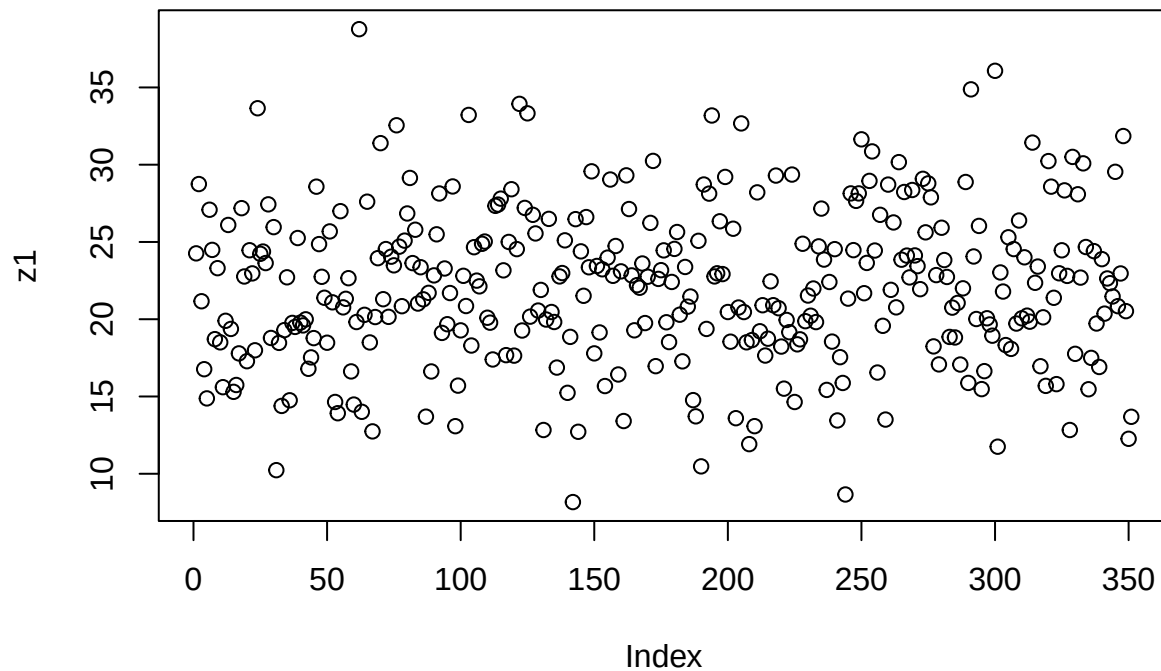
##Funciones Estadísticas

```
z1<-rnorm(351,22,5)
z1
```

```
## [1] 24.262405 28.747224 21.163853 16.766805 14.876493 27.086206 24.487769
## [8] 18.710771 23.305237 18.503251 15.602443 19.899141 26.101104 19.372543
## [15] 15.300647 15.744303 17.790205 27.188343 22.765737 17.284609 24.465294
## [22] 22.966979 17.989742 33.654230 24.234240 24.379730 23.635062 27.434231
## [29] 18.786630 25.960331 10.232455 18.470135 14.389889 19.294931 22.712884
## [36] 14.754040 19.762219 19.502421 25.254959 19.781104 19.600627 19.993950
## [43] 16.792355 17.531386 18.778690 28.579585 24.861198 22.746419 21.389622
## [50] 18.479139 25.683782 21.094246 14.645694 13.917162 26.993587 20.767374
## [57] 21.320390 22.656484 16.623305 14.478340 19.818213 38.770425 14.008062
## [64] 20.285381 27.607627 18.496773 12.742158 20.141290 23.952890 31.394581
## [71] 21.296670 24.540816 20.154174 24.049320 23.487479 32.551940 24.695647
## [78] 20.843904 25.086751 26.844827 29.143347 23.633349 25.791152 21.015464
## [85] 23.365437 21.287932 13.694712 21.706263 16.619638 22.837430 25.494999
## [92] 28.137667 19.114778 23.283591 19.677761 21.695154 28.591775 13.078601
## [99] 15.703184 19.278398 22.809502 20.854824 33.220477 18.298482 24.660712
## [106] 22.498887 22.127671 24.882243 25.028007 20.096676 19.776901 17.396167
## [113] 27.337943 27.419565 27.803760 23.163617 17.676238 25.001954 28.413758
## [120] 17.644477 24.537262 33.937755 19.270054 27.203578 33.323986 20.166755
## [127] 26.757158 25.551472 20.560481 21.897128 12.831902 19.978927 26.494891
## [134] 20.468630 19.821534 16.871222 22.769988 22.988615 25.104077 15.236425
## [141] 18.858238 8.167372 26.477503 12.716656 24.391918 21.517051 26.611389
## [148] 23.361282 29.571168 17.790228 23.446571 19.144082 23.232388 15.675957
## [155] 23.987029 29.041002 22.809592 24.739085 16.424621 23.097838 13.410631
## [162] 29.309901 27.131720 22.836095 19.282895 22.204185 22.037844 23.607427
## [169] 19.751175 22.750678 26.235060 30.245330 16.963170 22.608829 23.175307
## [176] 24.455559 19.804074 18.520800 22.413195 24.550107 25.643694 20.293511
## [183] 17.273452 23.381952 20.814388 21.470007 14.764624 13.715586 25.073526
```

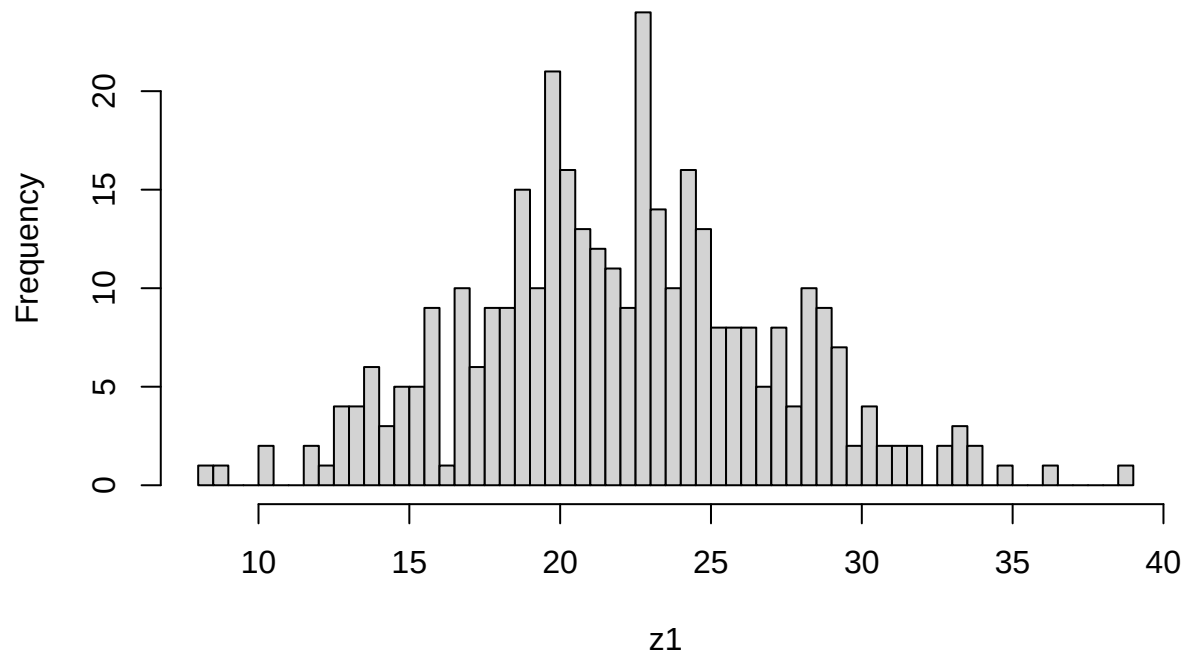
```
## [190] 10.477048 28.719163 19.372377 28.128168 33.184738 22.770058 22.972272
## [197] 26.338645 22.922342 29.208573 20.465954 18.546251 25.857425 13.596732
## [204] 20.756976 32.672568 20.463771 18.505537 11.916069 18.649372 13.079982
## [211] 28.208997 19.221884 20.906194 17.649190 18.740237 22.452984 20.907695
## [218] 29.296599 20.713391 18.230471 15.508081 19.952250 19.160023 29.362624
## [225] 14.643404 18.373912 18.699261 24.884281 19.873765 21.534084 20.226371
## [232] 21.987529 19.815259 24.714202 27.160436 23.861170 15.425881 22.409229
## [239] 18.552208 24.534808 13.449071 17.545189 15.875521 8.657844 21.333927
## [246] 28.141905 24.470933 27.660594 28.141794 31.641261 21.684611 23.643977
## [253] 28.953105 30.859759 24.446510 16.550383 26.756358 19.573410 13.506690
## [260] 28.713617 21.901911 26.262096 20.768750 30.166382 23.853560 28.233819
## [267] 24.117948 22.706315 28.363554 24.127432 23.436685 21.944780 29.086883
## [274] 25.622962 28.791254 27.887341 18.242447 22.855985 17.077339 25.920985
## [281] 23.829481 22.735929 18.853860 20.751512 18.820561 21.053796 17.068205
## [288] 21.999917 28.880091 15.877020 34.876156 24.061467 20.013504 26.038741
## [295] 15.477088 16.639379 20.062869 19.662073 18.934766 36.076395 11.750787
## [302] 23.027144 21.791577 18.327863 25.303512 18.081551 24.541397 19.720213
## [309] 26.389660 20.059370 24.007432 20.231851 19.841201 31.432090 22.358093
## [316] 23.407878 16.957371 20.125248 15.688310 30.233429 28.581346 21.388446
## [323] 15.798919 22.973783 24.460453 28.339102 22.804240 12.831736 30.503952
## [330] 17.759727 28.086267 22.693490 30.087761 24.673995 15.468297 17.497849
## [337] 24.407445 19.723208 16.908902 23.887588 20.362629 22.639522 22.332369
## [344] 21.472298 29.539328 20.846021 22.961995 31.853495 20.522607 12.262751
## [351] 13.686391
```

```
plot(z1)
```



```
hist(z1, main="Histograma de edades", breaks=50)
```

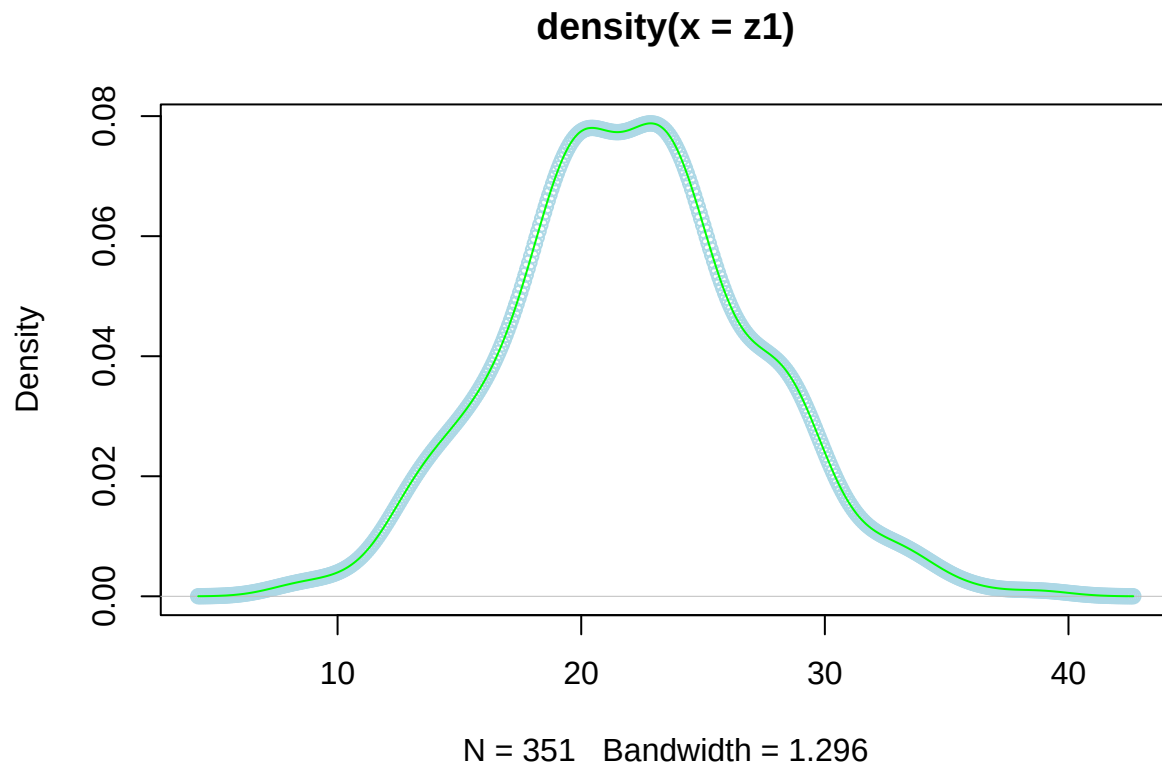
Histograma de edades



```
density(z1)
```

```
##  
## Call:  
## density.default(x = z1)  
##  
## Data: z1 (351 obs.); Bandwidth 'bw' = 1.296  
##  
##      x          y  
## Min.   : 4.278   Min.   :9.812e-06  
## 1st Qu.:13.874   1st Qu.:1.739e-03  
## Median :23.469   Median :1.327e-02  
## Mean   :23.469   Mean    :2.600e-02  
## 3rd Qu.:33.064   3rd Qu.:4.449e-02  
## Max.   :42.660   Max.    :7.880e-02
```

```
plot(density(z1),type="p",col="lightblue")  
lines(density(z1),col="green")
```

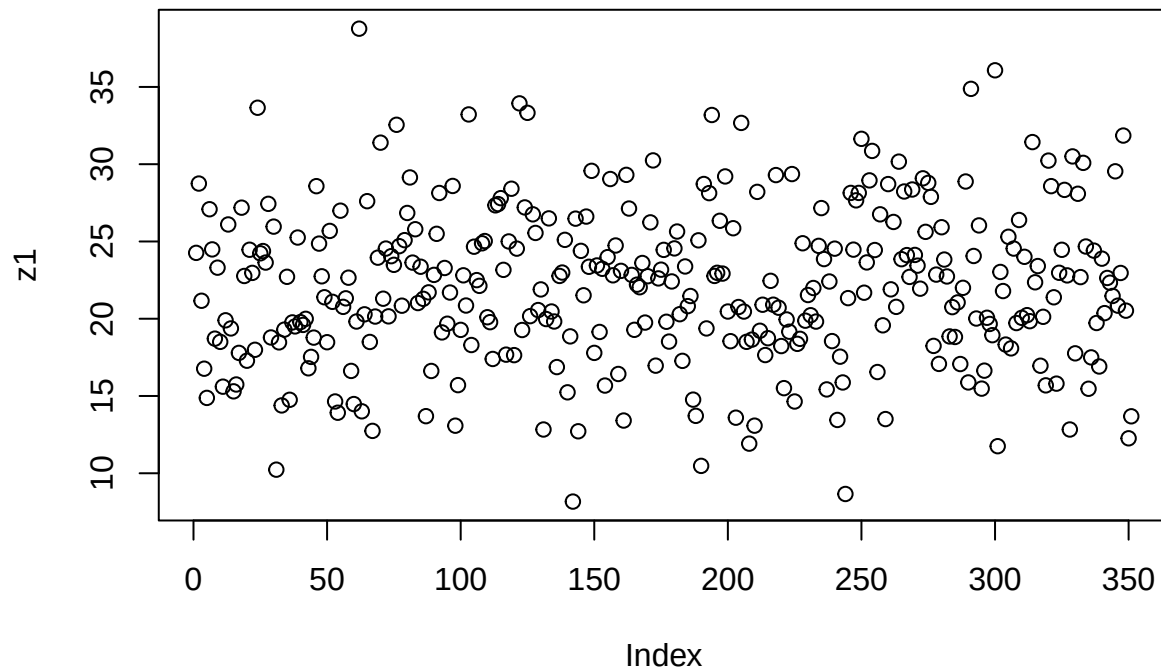


Normal {stats} R Documentation

The Normal Distribution

Description Density, distribution function, quantile function and random generation for the normal distribution with mean equal to mean and standard deviation equal to sd. Usage `dnorm(x, mean = 0, sd = 1, log = FALSE)` `pnorm(q, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `qnorm(p, mean = 0, sd = 1, lower.tail = TRUE, log.p = FALSE)` `rnorm(n, mean = 0, sd = 1)` Arguments x, q vector of quantiles. p vector of probabilities. n number of observations. If `length(n) > 1`, the length is taken to be the number required.

```
plot(z1)
```



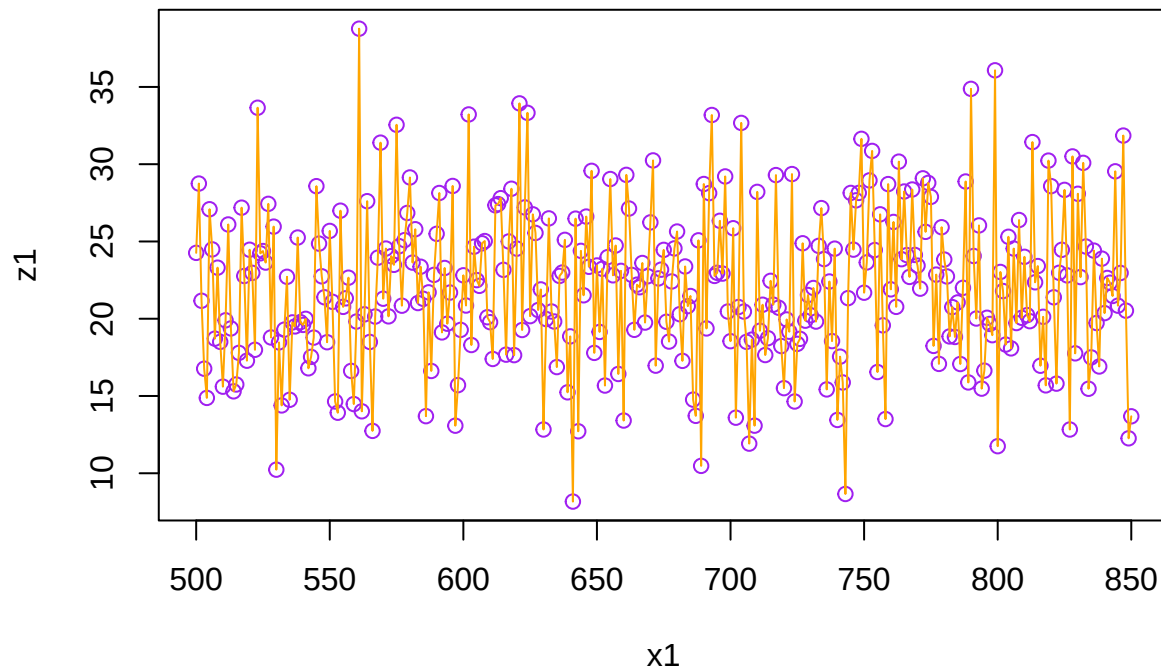
```
w1<-length(z1)
w1
```

```
## [1] 351
```

```
x1<-(500:850)
x1
```

```
## [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
## [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
## [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
## [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
## [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
## [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849 850
```

```
plot(x1, z1, col="purple")
lines(x1, z1, type="l", col="orange")
```



CLASE 8 DE ABRIL

Consigna 1: Las ultimas 3 cifras de mi DNI son 126, consigna crear una variable que tenga ese número.

```
DNI=126
```

consigna 2: Crear un vector que tenga todos los números desde el 1 hasta el 126 enteros.

```
for(i in 1:126)
  print(1:i)
```

```
## [1] 1
## [1] 1 2
## [1] 1 2 3
## [1] 1 2 3 4
## [1] 1 2 3 4 5
## [1] 1 2 3 4 5 6
## [1] 1 2 3 4 5 6 7
## [1] 1 2 3 4 5 6 7 8
## [1] 1 2 3 4 5 6 7 8 9
## [1] 1 2 3 4 5 6 7 8 9 10
## [1] 1 2 3 4 5 6 7 8 9 10 11
## [1] 1 2 3 4 5 6 7 8 9 10 11 12
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23
```

##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26																								
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27																							
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28																						
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29																					
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30																				
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31																			
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32																		
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33																	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34																
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34	35															
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34	35	36														
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34	35	36	37													

[illegible]

[illegible]

[illegible]

##	[26]	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
##	[51]	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75
##	[76]	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98		
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
##	[26]	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50
##	[51]	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75
##	[76]	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18							
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36							
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54							
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72							
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90							
##	[91]	91	92	93	94	95	96	97	98	99	100															
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18							
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36							
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54							
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72							
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90							
##	[91]	91	92	93	94	95	96	97	98	99	100	101														
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18							
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36							
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54							
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72							
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85												

##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109																	
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110																
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110	111															
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	3						

##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110	111	112	113	114	115											
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110	111	112	113	114	115	116										
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110	111	112	113	114	115	116	117									
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]	109	110	111	112	113	114	115	116	117	118								
##	[1]	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18
##	[19]	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36
##	[37]	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54
##	[55]	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72
##	[73]	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90
##	[91]	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108
##	[109]																		

```
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
## [1] 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18
## [19] 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
## [37] 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54
## [55] 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72
## [73] 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90
## [91] 91 92 93 94 95 96 97 98 99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
```

```
secuenciaDNI <- c()
for(i in 1:232){
  secuenciaDNI <- c(secuenciaDNI, i)
}
```

Consigna 3: Calcula la suma de todos los valores del vector secuenciaDNI con R

```
total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
  total<-total+secuenciaDNI[i]
}
print(total)
```

```
## [1] 27028
```

Consigna 4: Calcula la suma de todos los valores del vector secuenciaDNI con python.

```
secuenciaDNI = range(1, 233) # incluye 232
total = sum(secuenciaDNI)
print(total)
```

```
## 27028
```

Consigna 5: Calcula la suma de todos los valores del vector secuenciaDNI con Julia.No se programó para no ocupar memoria

Consigna 6: Determinar CPU time used.

```
require(stats)
system.time(for(i in 1:100) mad(runif(1000)))
```

```
## user system elapsed
```

```
##    0.012    0.000    0.011
require(stats)
system.time(for(i in 1:length(secuenciaDNI)){})
```

```
##      user  system elapsed
##    0.001    0.000    0.002

total<-0
valorfinal<-length(secuenciaDNI)
for (i in 1:valorfinal){
total<-total+secuenciaDNI[i]
}
print(total)
```

```
## [1] 27028

sleep_for_a_minute <- function() { Sys.sleep(14) }
start_time <- Sys.time()
sleep_for_a_minute()
end_time <- Sys.time()
end_time- start_time
```

```
## Time difference of 14.01196 secs
```

Consigna 7: Comparación de performance con system.time().

```
system.time({
  A <- numeric(50000)
  for (i in 1:50000) {
    A[i] <- i * 2
  }
})

##      user  system elapsed
##    0.004    0.000    0.004

system.time({
  B <- seq(from = 2, to = 100000, by = 2)
})
```

```
##      user  system elapsed
##    0.000    0.001    0.000
```

Biblioteca tictoc

Esto de usar una biblioteca es llamar u cargar una procedimientos que generará comando nuevos en R. Como ya fue comentado, cargar una biblioteca implica ejecutar el comando install.packages() o usar en r-studio el menú de Herramientas y Luego Instalar paquetes. Las funciones tic y toc son de la misma biblioteca de Octave/Matlab y se usan de la misma manera para la evaluación comparativa que el tiempo de sistema recién demostrado. Sin embargo, tictoc agrega mucha más comodidad al usuario y armonía al conjunto.

```
library(tictoc)
tic("sleeping")
A<-20
print("dormire una siestita...")
```

```
## [1] "dormire una siestita..."
```

```
## [1] "dormire una siestita..."
Sys.sleep(2)
print("...suena el despertador")
```

```
## [1] "...suena el despertador"
## [1] "...suena el despertador"
toc()
```

```
## sleeping: 2.004 sec elapsed
```

Biblioteca rbenchmark

La documentación de la función benchmark del paquete rbenchmark R lo describe como un simple contenedor alrededor de system.time. Sin embargo, agrega mucha conveniencia en comparación con las llamadas simples a system.time. Por ejemplo, requiere solo una llamada de referencia para cronometrar múltiples repeticiones de múltiples expresiones. Además, los resultados devueltos se organizan convenientemente en un marco de datos.

Implementación de una serie Fibonacci o Fibonacci

En matemáticas, la sucesión o serie de Fibonacci es la siguiente sucesión infinita de números naturales: 0, 1, 1, 2, 3, 5, 8 ... 89, 144, 233 ... La sucesión comienza con los números 0 y 1, 2 a partir de estos, cada término es la suma de los dos anteriores, es la relación de recurrencia que la define. A los elementos de esta sucesión se les llama números de Fibonacci. Esta sucesión fue descrita en Europa por Leonardo de Pisa, matemático italiano del siglo XIII también conocido como Fibonacci. Tiene numerosas aplicaciones en ciencias de la computación, matemática y teoría de juegos. Original de la Biblioteca Universidad de Florencia. Liber Abachi- Autor Fibonacci

Consigna 8: ¿Cuántas iteraciones se necesitan para generar un número de la serie mayor que 20 ? Además mostrar la serie de Fibonacci hasta el número obtenido anteriormente.

```
# Inicialización
a <- 0
b <- 1
serie <- c(a, b) # guardamos la serie
contador <- 2 # ya tenemos 2 términos
# Generar hasta superar 20
while (b <= 20) {
  siguiente <- a + b
  a <- b
  b <- siguiente

  serie <- c(serie, b) # agregamos a la serie
  contador <- contador + 1
}
# Resultados
cat("Cantidad de iteraciones:", contador, "\n")
```

```
## Cantidad de iteraciones: 9
```

```
cat("Primer número mayor a 20:", b, "\n")
```

```
## Primer número mayor a 20: 21
```

```
cat("Serie de Fibonacci hasta ese número:\n")
```

```
## Serie de Fibonacci hasta ese número:
```

```
print(serie)
```

```
## [1] 0 1 1 2 3 5 8 13 21
```

Consigna 9: Compara la performance de ordenación del método burbuja vs el método sort de R. Usar método microbenchmark para una muestra de tamaño 20.000

```
# Instalar paquete
```

```
if (!require(microbenchmark)) {  
  install.packages("microbenchmark")  
  library(microbenchmark)  
}
```

```
## Loading required package: microbenchmark
```

```
# Generar muestra
```

```
set.seed(123)
```

```
n <- 20000
```

```
vector <- sample(1:100000, n, replace = TRUE)
```

```
#Implementación de ordenamiento burbuja
```

```
bubble_sort <- function(x) {  
  n <- length(x)  
  for (i in 1:(n-1)) {  
    for (j in 1:(n-i)) {  
      if (x[j] > x[j+1]) {  
        temp <- x[j]  
        x[j] <- x[j+1]  
        x[j+1] <- temp  
      }  
    }  
  }  
  return(x)  
}
```

```
#Benchmark (OJO: burbuja es muy lento)
```

```
resultado <- microbenchmark(  
  burbuja = bubble_sort(vector),  
  sort_R = sort(vector),  
  times = 5  
)  
print(resultado)
```

```
## Unit: microseconds
```

```
##      expr      min       lq      mean     median      uq  
## burbuja 20929927.72 21003478.231 2.131558e+07 21038079.948 21469690.346  
## sort_R    651.78    769.171 8.115248e+02    786.021    786.381  
##      max neval  
## 22136725.582    5  
##   1064.271    5
```

La penitencia de Newton

Comentan algunos historiadores que Newton tenía en la escuela primaria un profesor muy exigente que reclamaba constantemente la atención de los alumnos a lo que el estaba enseñando. Tan pronto alguien cometía un acto impropio lo castigaba con una penitencia que consistía en traer para el día siguiente el resultado de la suma desde el número 1 hasta cierto número aleatoriamente elegido. Por supuesto esta tarea demandaría quedarse toda la noche haciendo las sumas parciales, tarea que el profesor ya había realizado

previamente y que tenía registrado en un voluminoso cuaderno que (al estilo tabla de logaritmos) cargaba religiosamente en su portafolios de cuero.

Consigna 10: Desarrolla dos algoritmos que hagan este trabajo, por ejemplo para sumar desde 1 hasta 1×1010 y verifica cuál de los dos es más eficiente.

```
#Algoritmo 1: suma con for (iterativo)
```

```
n <- 1010
suma1 <- 0
for (i in 1:n){
  suma1 <- suma1 + i
}
suma1
```

```
## [1] 510555
```

```
#Complejidad: O(n) (suma uno por uno)
```

```
#Algoritmo 2: fórmula matemática (Gauss)
```

```
n <- 1010
suma2 <- n * (n + 1) / 2
suma2
```

```
## [1] 510555
```

```
#Complejidad: O(1) (una sola operación)
```

```
#Comparación de eficiencia: medir tiempos con system.time():
```

```
# Método 1
```

```
system.time({
  suma1 <- 0
  for (i in 1:1010){
    suma1 <- suma1 + i
  }
})
```

```
##      user  system elapsed
##    0.002   0.000   0.001
```

```
# Método 2
```

```
system.time({
  suma2 <- 1010 * (1010 + 1) / 2
})
```

```
##      user  system elapsed
##         0         0         0
```

Se desarrollaron dos algoritmos para sumar desde 1 hasta 1×1010 . El método iterativo presenta complejidad lineal, lo que lo hace ineficiente para valores grandes como 10.000.000.000. En cambio, el uso de la fórmula cerrada reduce la complejidad a constante, permitiendo obtener el resultado de forma inmediata. Por lo tanto, el segundo algoritmo es significativamente más eficiente.

##Matriz Inversa y Gráficos Violín

En este apartado se realiza una comparación del rendimiento computacional de distintos métodos para estimar los coeficientes de un modelo de regresión lineal en R. Para ello, se emplea la librería microbenchmark, la cual permite medir con precisión los tiempos de ejecución de diferentes algoritmos a través de múltiples repeticiones.

Se genera un conjunto de datos simulado utilizando funciones de generación aleatoria, compuesto por una matriz de variables explicativas X de dimensión 10000×100 y un vector respuesta y, construido como una

combinación lineal de X con un término de error. Esto permite trabajar en un contexto controlado pero representativo de problemas de gran escala.

A continuación, se implementan tres enfoques para el cálculo de los coeficientes: el método clásico mediante la función `lm()`, el cálculo explícito usando la inversa de la matriz y la resolución directa del sistema lineal equivalente mediante la función `solve()`. Para asegurar la validez de la comparación, se define una función que verifica que los coeficientes obtenidos por los distintos métodos sean numéricamente equivalentes dentro de una tolerancia establecida.

Finalmente, se visualizan los resultados utilizando la librería `ggplot2`, a través de la función `autoplot()`, que permite representar gráficamente la distribución de los tiempos de ejecución de cada método, facilitando su análisis comparativo.

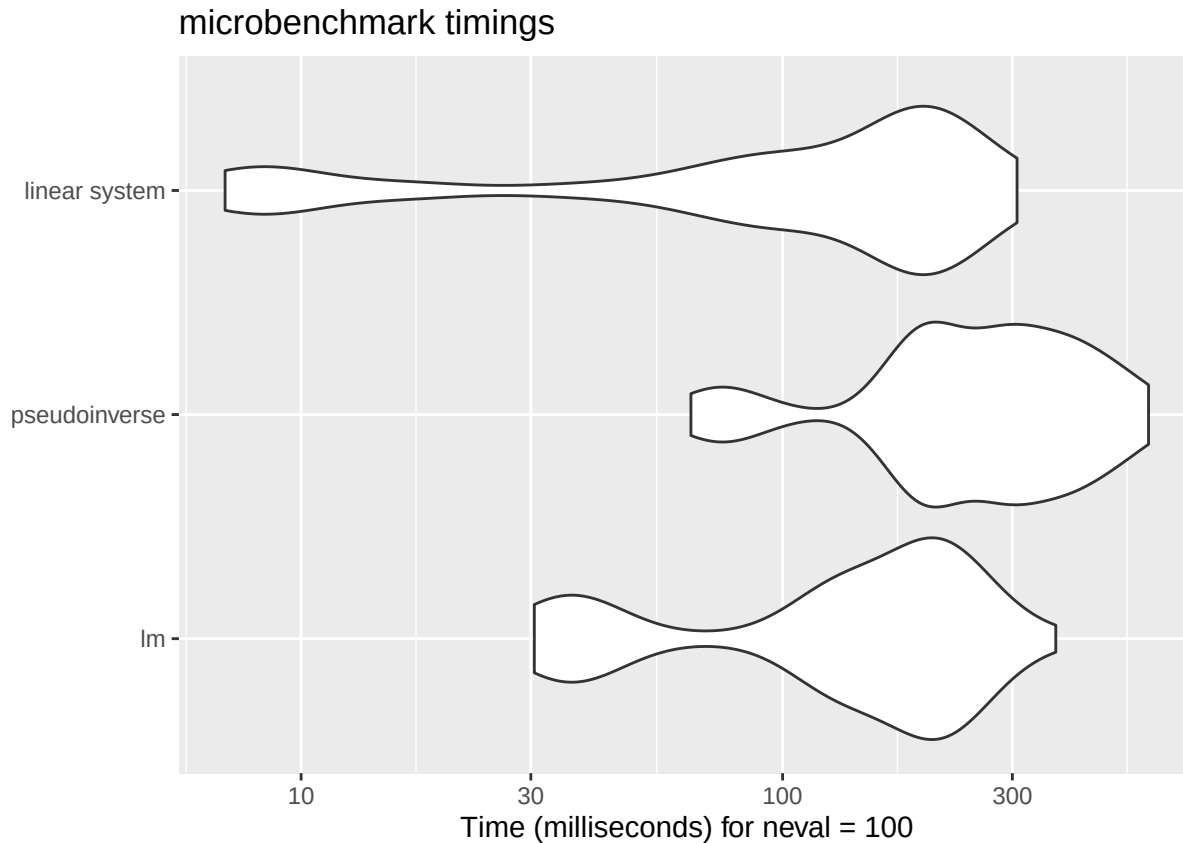
```
library(microbenchmark)
set.seed(2017)
n <- 10000
p <- 100
X <- matrix(rnorm(n * p), n, p)
y <- X %>% rnorm(p) + rnorm(100)
check_for_equal_coefs <- function(values) {
  tol <- 1e-12
  max_error <- max(
    c(
      abs(values[[1]] - values[[2]]),
      abs(values[[2]] - values[[3]]),
      abs(values[[1]] - values[[3]])
    )
  )
  max_error < tol
}
mbm <- microbenchmark(
  "lm" = {
    b <- lm(y ~ X + 0)$coef
  },
  "pseudoinverse" = {
    b <- solve(t(X) %>% X) %>% t(X) %>% y
  },
  "linear system" = {
    b <- solve(t(X) %>% X, t(X) %>% y)
  },
  check = check_for_equal_coefs
)
mbm

## Unit: milliseconds
##      expr      min       lq      mean    median      uq      max neval
##      lm 30.504845  88.66422 153.2278 153.6236 215.0404 369.2012   100
## pseudoinverse 64.505572 200.71528 279.4820 282.4377 385.9614 575.3349   100
## linear system  6.952626  68.45507 133.2611 168.5937 198.9147 306.7988   100

library(ggplot2)
autoplot(mbm)

## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the microbenchmark package.
```

```
## Please report the issue at
## <https://github.com/joshuaulrich/microbenchmark/issues/>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



La línea: `autoplot(mbm)` genera automáticamente un gráfico a partir del objeto de microbenchmark, y por defecto ese gráfico es un boxplot (diagrama de cajas), que muestra: mediana, cuartiles, dispersión de los tiempos. No es un gráfico de violín. Entonces, este gráfico que sigue es mucho mejor que solo boxplot. El violín deja ver si hay: colas largas, multimodalidad y variabilidad real.

```
library(ggplot2)
autoplot(mbm) +
  geom_violin(fill = "darkblue", alpha = 0.4, color = NA) + # distribución
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.7) + # resumen
  stat_summary(fun = mean, geom = "point",
    shape = 23, size = 3, fill = "pink") + # media
  labs(
    title = "Comparación de tiempos de ejecución",
    x = "Método",
    y = "Tiempo (ms)"
  ) +
  theme_minimal() +
  theme(
    legend.position = "none",
    plot.title = element_text(hjust = 0.5, face = "bold")
  )
```

Comparación de tiempos de ejecución

